

The University of Faisalabad

Department of Computer Science



The
University of
Faisalabad

Name: Humna Imran

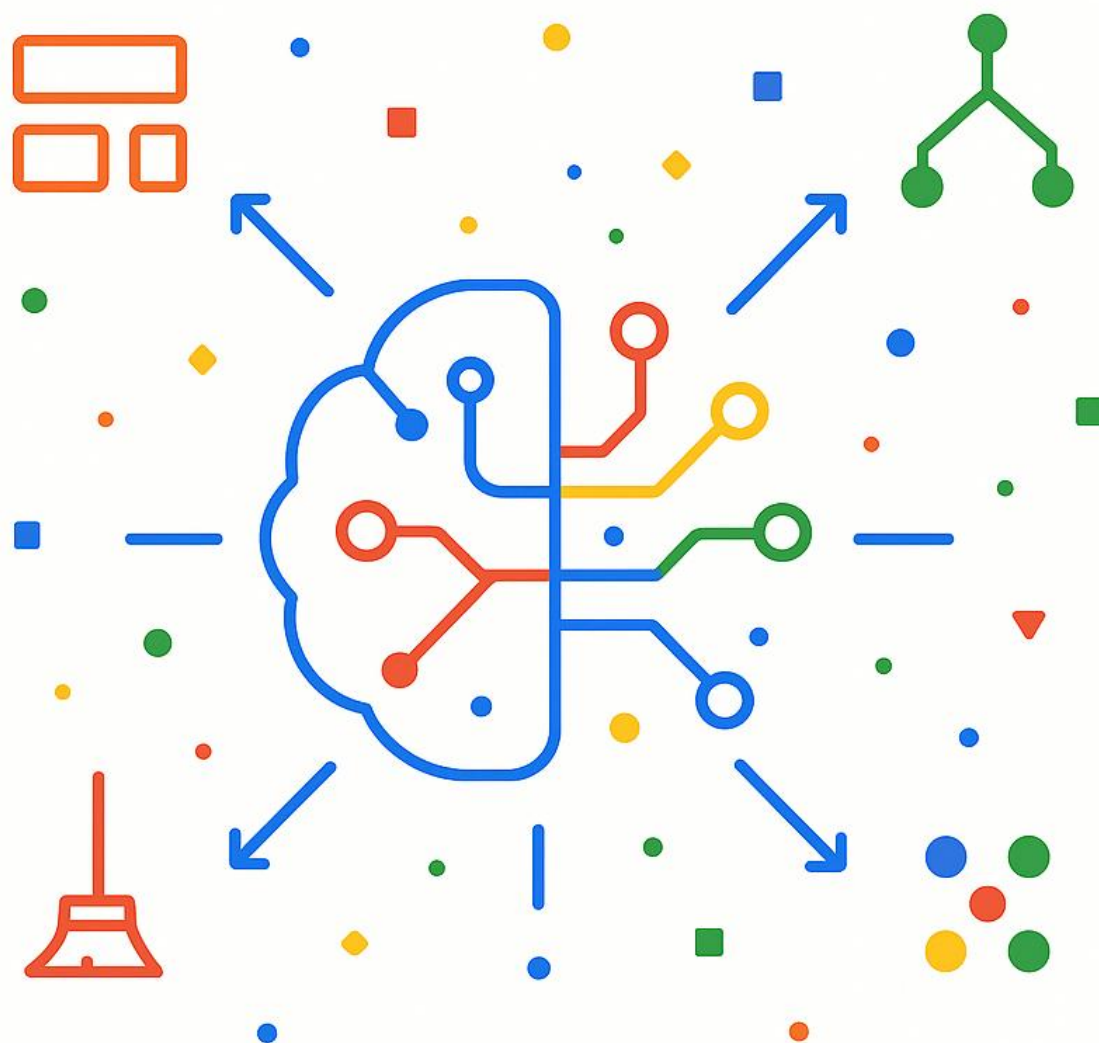
Registration Number: 2023-BS-AI-017

Instructor: Khadija Lateef

Natural Language Processing

SE-314

BS Artificial Intelligence



Natural Language Processing Lab Manual

Fundamentals, Concepts, and Hands-on Exercises

Table of Contents

Lab 1: Introduction to Python.....	1
Lab 2: Text Preprocessing	1
Lab 3: Regular Expressions	4
Lab 4: Tokenization	8
Lab 5: Stemming and Lemmatization.....	13
Lab 6: Text Embedding	18
Lab 7: Parsing	22
Lab 8: Naïve Bayes Classification.....	27
Lab 11: Named Entity Recognition (NER) Using HMMs	31
Lab 12: POS Tagging Using HMMs	35
Lab 14: Context-Free Grammars (CFGs).....	39
Lab 15: Probabilistic CFGs Using The Penn Treebank	43

Lab 1: Introduction to Python

Overview

This lab introduces fundamental Python programming concepts essential for Natural Language Processing, including list manipulation, iteration, arithmetic operations, and defining custom functions.



1. List Functions

Lists are mutable sequences used to store collections of items. This task demonstrates defining a list and printing its contents.

Input Code

```
print("List Functions")
Shopping_list = ['Apple', 'Banana', 'Mango', 'Orange', 'Pineapple',
                'Guava']
print("List of fruits include: ", Shopping_list)
```

Output

```
List Functions
List of fruits include: ['Apple', 'Banana', 'Mango',
                        'Orange', 'Pineapple', 'Guava']
```

2. Loops

Loops allow for iterating over data structures. The following example demonstrates iterating through the Shopping_list to process each item individually.

Input Code

```
print("\nLoop of fruits")
for i in Shopping_list:
    print(i, "is in the list")
```

Output

```
Loop of fruits
Apple is in the list
Banana is in the list
Mango is in the list
Orange is in the list
Pineapple is in the list
Guava is in the list
```

3. Arithmetic Operations

Python serves as a powerful calculator. This task accepts user input to perform basic arithmetic: addition, subtraction, multiplication, and division.

Input Code

```
print("\nArithmetic Operations")
## Arithmetic Operations (Task 2)
a = int(input("Enter First Number: "))
b = int(input("Enter First Number: ")) # Note: Label should be 'Second
Number' based on context

print("Sum: ", a+b)
print("Sub: ", a-b)
print("Mul: ", a*b)
print("Div: ", a/b)
```

Output

```
Arithmetic Operations
Sum: 5
Sub: -1
Mul: 6
Div: 0.6666666666666666
```

4. Functions

Functions encapsulate reusable logic. This section covers three specific tasks: printing a name, checking for even/odd numbers, and finding the maximum of two numbers.

A. Simple Name Function

Input Code

```
print("\nSimple Name Function")
def printname():
    print("Humna Imran")

printname()
```

Output

```
Simple Name Function
Humna Imran
```

B. Even Odd Function

Input Code

```
print("\nEven Odd Function")
def even_or_odd(x):
    if(x%2==0):
        print(x, "is Even Number")
    else:
        print(x, "is Odd Number")

number = int(input("Input Any Number: "))
even_or_odd(number)
```

Output

```
Even Odd Function
4 is Even Number
```

C. Min Max Function

Input Code

```
print("\nMin Max Function")
def max(x,y):
    if(x>y):
        print(x, "is Maximum")
    else:
        print(y, "is Maximum")

number1 = int(input("Input First Number: "))
number2 = int(input("Input Second Number: "))
max(number1, number2)
```

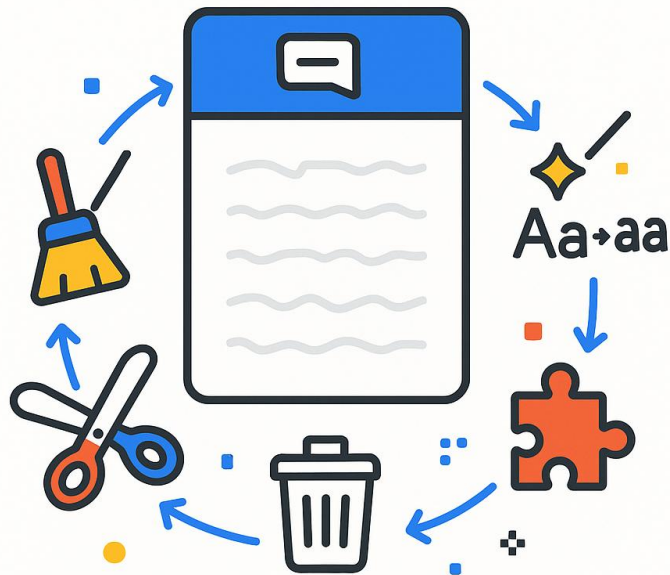
Output

Min Max Function
7 is Maximum

Lab 2: Text Preprocessing

Overview

Text preprocessing is a critical step in Natural Language Processing (NLP) that transforms raw text into a clean, machine-readable format. This lab focuses on standard preprocessing techniques including lowercasing, noise removal (numbers and punctuation), and stopwords filtering.



1. Importing Libraries

We begin by importing the necessary modules: `re` for regular expressions, `string` for string manipulation, and `nltk` for natural language data handling.

Input Code

```
import re
import string
import nltk
from nltk.corpus import stopwords
```

2. Corpus Definition

The raw text (corpus) serves as the input data. In this example, the corpus describes the University of Faisalabad.

Input Code

```
corpus = (  
    "The University of Faisalabad is a private university located in  
    Faisalabad, Pakistan. "  
    "It was established in 2002 by the Madina Foundation. "  
    "The university offers programs in engineering, health sciences,  
    management, and social sciences. "  
    "TUF is recognized by the Higher Education Commission of Pakistan. "  
    "The university has modern laboratories, libraries, and research  
    facilities. "  
    "It focuses on producing skilled graduates to meet industry and societal  
    needs. "  
    "TUF places strong emphasis on research, innovation, and community  
    service. "  
    "The university encourages extracurricular activities, seminars, and  
    workshops for student development. "  
    "It has collaborations with national and international institutions. "  
    "The University of Faisalabad continues to grow as a center of excellence  
    in education and research."  
)
```

3. Preprocessing Logic

This section implements a pipeline to clean the text data through four specific transformations:

1. **Lowercasing:** Converts all characters to lowercase to ensure consistency.
2. **Number Removal:** Uses regular expressions to strip numerical digits.
3. **Punctuation Removal:** Uses a translation table to remove special characters.
4. **Stopword Removal:** Filters out common words (like "is", "the", "in") that carry minimal semantic meaning.

Input Code

```
# Lower Corpus  
corpus = corpus.lower()  
  
# Remove Numbers  
corpus = re.sub(r"\d+", "", corpus)  
  
# Remove punctuation  
translator = str.maketrans("", "", string.punctuation)
```

```
corpus = corpus.translate(translator)

# Remove stopwords
stop_words = set(stopwords.words("english"))
words = corpus.split()
filtered_words = [w for w in words if w not in stop_words]

# Join back into a sentence
cleaned_text = " ".join(filtered_words)
```

4. Output

The final output is a refined string containing only the semantically significant words.

Input Code

```
# Print Cleaned Data
print(cleaned_text)
```

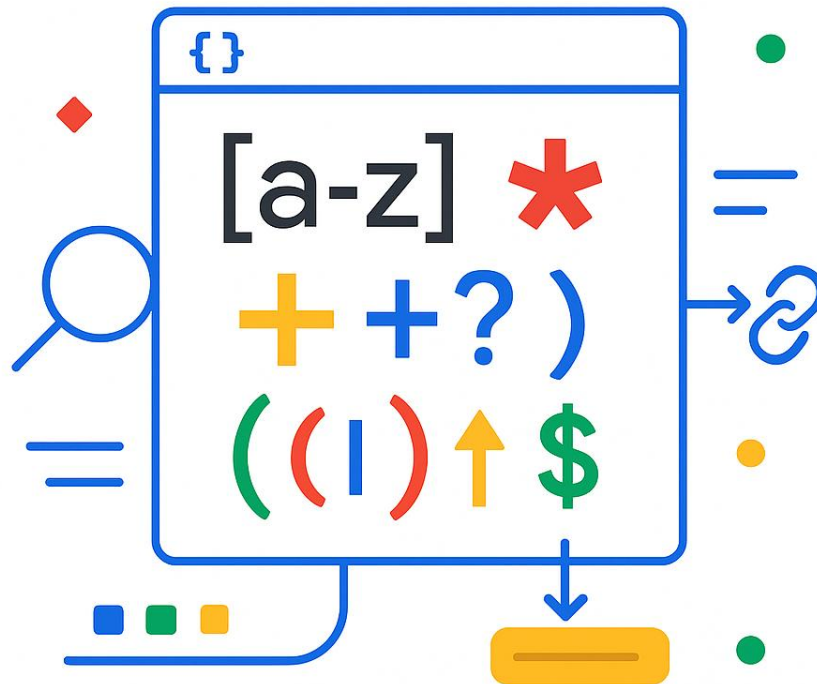
Output

```
university faisalabad private university located faisalabad
pakistan established madina foundation university offers
programs engineering health sciences management social
sciences tuf recognized higher education commission pakistan
university modern laboratories libraries research facilities
focuses producing skilled graduates meet industry societal
needs tuf places strong emphasis research innovation
community service university encourages extracurricular
activities seminars workshops student development
collaborations national international institutions university
faisalabad continues grow center excellence education
research
```

Lab 3: Regular Expressions

Overview

Regular expressions (regex) are sequences of characters that define a search pattern. This lab explores the `re` module in Python, demonstrating key functions for pattern matching, searching, and string manipulation.



Regular Expressions

1. Find All Matches

The `re.findall()` function returns all non-overlapping matches of a pattern in a string as a list of strings.

Input Code

```
# 1. re.findall()
text1 = "Dog, Cat, Dog, Bird, Dog"
```

```
result1 = re.findall(r"Dog", text1)
print("findall:", result1)
```

Output

```
findall: ['Dog', 'Dog', 'Dog']
```

2. Compile Pattern

`re.compile()` compiles a regular expression pattern into a regex object, which can be used for matching using its `match()`, `search()`, and other methods. This is efficient when the same pattern is used multiple times.

Input Code

```
pattern = re.compile(r"\d{2}/\d{2}/\d{4}")
text2 = "The event is on 05/21/2025"
result2 = pattern.search(text2)
print("compile:", result2.group())
```

Output

```
compile: 05/21/2025
```

3. Split String

The `re.split()` function splits a string by the occurrences of the pattern. It is more flexible than the standard string `split()` method because it can split by complex patterns.

Input Code

```
text3 = "car;bus|train,plane"
result3 = re.split(r"[;|]", text3)
print("split:", result3)
```

Output

```
split: ['car', 'bus', 'train', 'plane']
```

4. Substitution with Count

`re.subn()` performs the same operation as `re.sub()`, but it returns a tuple containing the new string and the number of substitutions made.

Input Code

```
text4 = "one two one three one four"
result4 = re.subn(r"one", "zero", text4)
print("subn:", result4)
```

Output

```
subn: ('zero two zero three zero four', 3)
```

5. Escape Special Characters

`re.escape()` escapes special characters in a pattern. This is useful when you want to match a string that may contain regex metacharacters literally.

Input Code

```
text5 = "Price is $5.99 (discount)"
escaped = re.escape(text5)
print("escape:", escaped)
```

Output

```
escape: Price\ is\ $5.99\ (discount)
```

6. Search Pattern

`re.search()` scans through a string, looking for the first location where the regular expression pattern produces a match. It returns a match object if found, otherwise `None`.

Input Code

```
text6 = "Coding with Python is powerful!"
result6 = re.search(r"Python", text6)
```

```
print("search:", result6.group())
```

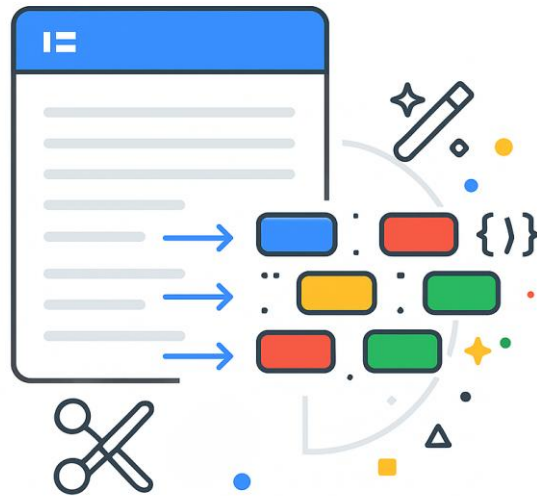
Output

```
search: Python
```

Lab 4: Tokenization

Overview

Tokenization is the process of breaking down text into smaller units called tokens, such as words or sentences. This lab explores different tokenization techniques using Python libraries (NLTK, SpaCy) and regular expressions, analyzing their performance and handling of special cases.



Tokenization

1. Importing Libraries & Corpus Setup

We begin by importing the required libraries and downloading the necessary NLTK data packages (punkt, punkt_tab).

Input Code

```
import nltk
import re
import string
import spacy
from nltk.tokenize import word_tokenize, sent_tokenize
from nltk.probability import FreqDist
```

```
# Download necessary datasets
nltk.download('punkt')
nltk.download('punkt_tab')

# Define Corpus
text = """Prof. Johnson completed his PhD at Stanford University in 2015.
He now works at Google Research developing machine learning models.
Contact him at prof.johnson@gmail.com or visit https://research.google.com for
more info!"""
```

2. Word Tokenization (NLTK)

Word tokenization splits text into individual words. NLTK's `word_tokenize` handles punctuation as separate tokens.

Input Code

```
print("----- TASK 2: Word Tokenization using NLTK -----")

tokens = word_tokenize(text)
print("Tokens:", tokens)
print("\nTotal Tokens:", len(tokens))
print("Unique Tokens:", len(set(tokens)))

# Frequency distribution
freq = FreqDist(tokens)
print("\nToken Frequencies:")
for word, count in freq.items():
    print(f"{word}: {count}")
```

Output

```
Tokens: ['Prof.', 'Johnson', 'completed', 'his', 'PhD', 'at',
'Stanford', ...]
Total Tokens: 37
Unique Tokens: 34
```

3. Sentence Tokenization (NLTK)

Sentence tokenization splits text into sentences, which is useful for processing paragraph-level

data.

Input Code

```
print("\n----- TASK 3: Sentence Tokenization using NLTK -----")

sentences = sent_tokenize(text)
print("Sentences:", sentences)
print("Total Sentences:", len(sentences))

# Analyze sentence lengths
longest = max(sentences, key=len)
shortest = min(sentences, key=len)
print("\nLongest Sentence:", longest)
print("Shortest Sentence:", shortest)
```

Output

```
Sentences: ['Prof. Johnson completed his PhD...', 'He now
works...', 'Contact him...']
Total Sentences: 3
```

4. Tokenization using Regular Expressions

Regex allows for custom tokenization rules, such as removing punctuation entirely or splitting by specific patterns.

Input Code

```
print("\n----- TASK 4: Tokenization using Regular Expressions -----")

# Remove punctuation and split on whitespace
regex_tokens = re.findall(r'\b\w+\b', text)
print("Regex Tokens:", regex_tokens)
print("\nTotal Regex Tokens:", len(regex_tokens))

# Comparison
print("\nObservation: Regex removed punctuation completely, while NLTK
preserved symbols like '.' and '-'.")
print("Hence, NLTK handled punctuation contextually better.")
```

5. Tokenization using SpaCy

SpaCy provides an industrial-strength tokenizer that is often faster and handles complex entity types (like URLs and emails) more effectively than NLTK.

Input Code

```
print("\n----- TASK 5: Tokenization using SpaCy -----")

nlp = spacy.load("en_core_web_sm")
doc = nlp(text)

spacy_tokens = [token.text for token in doc]
print("SpaCy Tokens:", spacy_tokens)

print("\nComparison:")
print("NLTK handled abbreviations & punctuation well, but SpaCy handled email
and URL tokens better.")
```

6. Custom Tokenizer Function

Creating a manual tokenizer allows for full control over preprocessing steps, such as lowercasing and character filtering.

Input Code

```
print("\n----- TASK 6: Custom Tokenizer Function -----")

def custom_tokenizer(text):
    text = text.lower()
    text = text.translate(str.maketrans('', '', string.punctuation))
    return text.split()

custom_tokens = custom_tokenizer(text)
print("Custom Tokens:", custom_tokens)
print("\nTotal Tokens:", len(custom_tokens))
print("\nComparison: Custom tokenizer is simple but removes punctuation and
capitalization info.")
```

7. Subword Tokenization (Advanced)

Subword tokenization splits words into smaller units (e.g., syllables or morphemes), which is crucial for modern Large Language Models (LLMs) to handle unknown words.

Input Code

```
print("\n----- TASK 7: Subword Tokenization (Advanced) -----")

# Using NLTK's Syllable Tokenizer approximation via Regex
words = ["processing", "tokenization", "university"]

def simple_syllable_split(word):
    return re.findall(r'^[aeiou]*[aeiou]+', word.lower())

for w in words:
    print(f"{w} -> {simple_syllable_split(w)}")
```

Output

```
processing -> ['pro', 'ce', 'ssi']
tokenization -> ['to', 'ke', 'ni', 'za', 'tio']
```

8. Evaluation and Comparison

The table below summarizes the trade-offs between different tokenization methods.

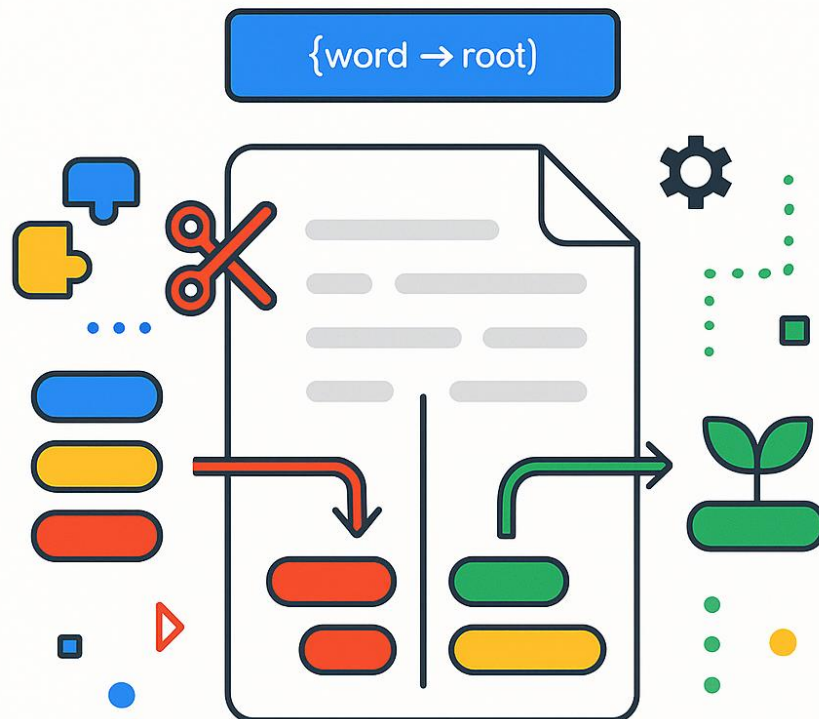
Method	Handles Punctuation	Handles Email	Handles URL	Preserves Case	Ease of Use
NLTK	Contextual	Partial	Partial	Yes	Easy
Regex	Removes	Breaks	Breaks	No	Medium
SpaCy	Excellent	Yes	Yes	Yes	Medium-Easy
Custom	Removes	Breaks	Breaks	No	Very Easy

Lab 5: Stemming and Lemmatization

Overview

This lab explores text normalization techniques in NLP: **Stemming** and **Lemmatization**.

- **Stemming** is a crude heuristic process that chops off the ends of words.
- **Lemmatization** uses a vocabulary and morphological analysis to return the base or dictionary form of a word (lemma).



Stemming & Lemmatization

1. Setup and Imports

We utilize NLTK for traditional lemmatization and spaCy for a more modern, efficient approach. We also use pandas for data handling and matplotlib for visualization.

Input Code

```
import nltk
```

```

from nltk.stem import WordNetLemmatizer, PorterStemmer
from nltk.corpus import wordnet, stopwords
import spacy
from collections import Counter
import pandas as pd
import matplotlib.pyplot as plt

# Load SpaCy model and Initialize NLTK Lemmatizer
nlp = spacy.load('en_core_web_sm')
lemmatizer = WordNetLemmatizer()

```

2. NLTK Lemmatization with POS Tags

NLTK's WordNetLemmatizer requires Part-of-Speech (POS) tags to accurately resolve words (e.g., treating "better" as an adjective vs. a verb). The helper function below maps NLTK tags to WordNet tags.

Input Code

```

print("----- Task 1: Basic NLTK Lemmatization -----")

def get_wordnet_pos(treebank_tag):
    if treebank_tag.startswith('J'):
        return wordnet.ADJ
    elif treebank_tag.startswith('V'):
        return wordnet.VERB
    elif treebank_tag.startswith('N'):
        return wordnet.NOUN
    elif treebank_tag.startswith('R'):
        return wordnet.ADV
    else:
        return wordnet.NOUN # Default

words = ["studies", "am", "went", "feet", "flying", "cars", "mice",
"better", "ate"]
pos_tags = nltk.pos_tag(words)

for word, tag in pos_tags:
    lemma = lemmatizer.lemmatize(word, get_wordnet_pos(tag))
    print(f"{word} ({tag}) -> {lemma}")

```

Output

```
studies (NNS) -> study
am (VBP) -> be
went (VBD) -> go
feet (NNS) -> foot
...
better (RB) -> well
ate (VB) -> eat
```

3. Comparison: Stemming vs. Lemmatization

This task highlights the difference in output quality. Stemmers (like PorterStemmer) often produce non-words ("studi"), while lemmatizers produce valid dictionary words ("study").

Input Code

```
print("\n----- Task 3: Stemming vs Lemmatization -----")

stemmer = PorterStemmer()
words = ["studies", "better", "flying", "easily", "running"]

print(f"{'Word':<10}{{'Stem':<15}{{'Lemma':<15}}}")
for w in words:
    stem = stemmer.stem(w)
    # Note: Using default noun context for simple lemmatization demo
    lemma = lemmatizer.lemmatize(w,
get_wordnet_pos(nltk.pos_tag([w])[0][1]))
    print(f"w:<10}{stem:<15}{lemma:<15}")
```

Output

```
Word Stem Lemma
studies studi study
better better well
flying fli fly
easily easili easily
running run run
```

4. Advanced Lemmatization Pipeline (spaCy)

SpaCy offers a streamlined pipeline that handles tokenization, POS tagging, and lemmatization automatically. It also handles stopword removal efficiently.

Input Code

```
print("\n----- Task 6 & 7: SpaCy Lemmatization & Comparison -----")

sentence = "The students are studying hard for their final examinations."
stop_words = set(stopwords.words('english'))
doc = nlp(sentence)

# Extract lemmas for non-stop, alpha words
clean_tokens = [
    token.lemma_
    for token in doc
    if token.text.lower() not in stop_words and token.is_alpha
]

print("Original Sentence:", sentence)
print("Clean Lemmas:", clean_tokens)
```

Output

```
Original Sentence: The students are studying hard for their
final examinations.10
Clean Lemmas: ['student', 'study', 'hard', 'final',
'examination']
```

5. Custom Lemmatizer Class

For production environments, encapsulating logic into a class ensures reusability. This custom class uses spaCy to lemmatize text and compares similarity between sentences based on lemma overlap.

Input Code

```
print("\n----- Task 12: Custom Lemmatizer Class -----")

class MyLemmatizer:
    def __init__(self):
        self.nlp = spacy.load("en_core_web_sm")
```

```
def lemmatize(self, text):
    doc = self.nlp(text)
    return [t.lemma_ for t in doc if t.is_alpha]

def compare(self, s1, s2):
    lemmas1, lemmas2 = set(self.lemmatize(s1)),
set(self.lemmatize(s2))
    overlap = len(lemmas1 & lemmas2) / len(lemmas1 | lemmas2) * 100
    print(f"Lemma Overlap: {overlap:.2f}%")

my_lem = MyLemmatizer()
my_lem.compare("Students are learning AI.", "Students learned AI in class.")
```

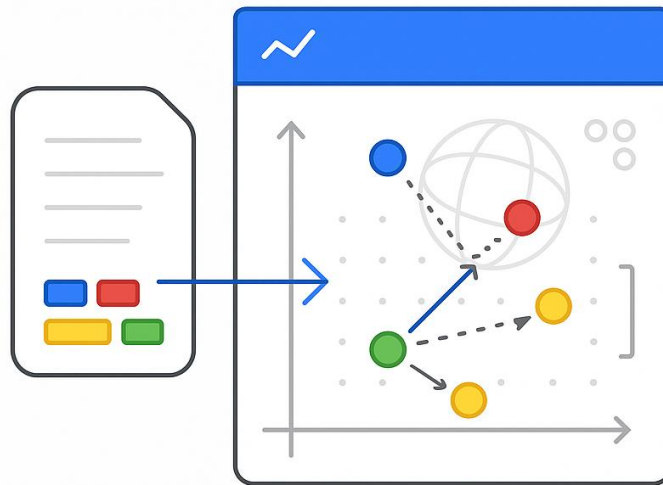
Output

Lemma Overlap: 50.00%

Lab 6: Text Embedding

Overview

Text embedding is a technique in Natural Language Processing (NLP) that converts text into numerical vectors. These vectors capture semantic meaning, allowing similar texts to be represented by vectors that are close to each other in a high-dimensional space. This enables computers to perform tasks like semantic search and similarity analysis.



Text Embedding

1. Setup and Dataset

We import the necessary libraries, including `gensim` for training embeddings and `numpy` for vector operations. We define a small toy dataset to demonstrate the concepts.

Input Code

```
import numpy as np
from gensim.models import Word2Vec, FastText
```

```

from numpy.linalg import norm

# Define Corpus
sentences = [
    "I love playing football on weekends.",
    "Football players train every day.",
    "Machine learning is fun and powerful.",
    "I enjoy working with neural networks and embeddings.",
    "The weather today is sunny and warm.",
    "Cats and dogs are common pets.",
    "I went to the bank to deposit money.",
    "The river bank was full of flowers."
]

# Tokenization
tokenized = [s.lower().split() for s in sentences]

```

2. Task 1: Word2Vec Skip-Gram

We train a **Word2Vec** model using the Skip-Gram architecture. This predicts context words given a target word, effectively learning semantic relationships.

Input Code

```

print("\n=== TASK 1: Word2Vec Skip-Gram ===")
# Train Word2Vec model
w2v = Word2Vec(sentences=tokenized, vector_size=100, window=5,
min_count=1, sg=1, epochs=20)

word = "football"
if word in w2v.wv:
    print(f"Top 10 words similar to '{word}':")
    print(w2v.wv.most_similar(word, topn=10))

```

3. Task 2: Semantic Similarity (GloVe/Word2Vec)

This task calculates the cosine similarity between word vectors to measure how closely related two words are semantically.

Input Code

```

print("\n=== TASK 2: Semantic Similarity ===")

def cos_sim(a, b):
    return float(np.dot(a, b) / (norm(a) * norm(b) + 1e-9))

pairs = [("king", "queen"), ("cat", "dog"), ("football", "soccer"), ("car",
"bicycle")]

for a, b in pairs:
    # Note: Using our small trained w2v model for demonstration
    if a in w2v.wv and b in w2v.wv:
        similarity = cos_sim(w2v.wv[a], w2v.wv[b])
        print(f"{a} - {b}: {similarity:.4f}")
    else:
        print(f"{a} or {b} not in vocabulary.")

```

4. Task 3: FastText vs. Word2Vec (OOV Handling)

Standard Word2Vec fails when it encounters Out-Of-Vocabulary (OOV) words. **FastText** solves this by breaking words into character n-grams, allowing it to generate embeddings for unseen words based on their subword components.

Input Code

```

print("\n=== TASK 3: FastText vs Word2Vec OOV ===")

# Train FastText model
ft = FastText(sentences=tokenized, vector_size=100, window=5,
min_count=1, epochs=20)

oov_words = ["footbal", "footballer", "neuralnetwork", "unknownword"]

for w in oov_words:
    print(f"fastText('{w}') similarity to 'football':
{ft.wv.similarity('football', w):.4f}")

    try:
        print(f"word2vec('{w}') similarity: {w2v.wv.similarity('football',

```

```
w)}")  
    except KeyError:  
        print(f"Word2Vec: '{w}' not in vocabulary")
```

5. Task 4: Sentence Embeddings

We can create a simple embedding for a full sentence by averaging the vectors of its constituent words. This is often used as a baseline for classification tasks like sentiment analysis.

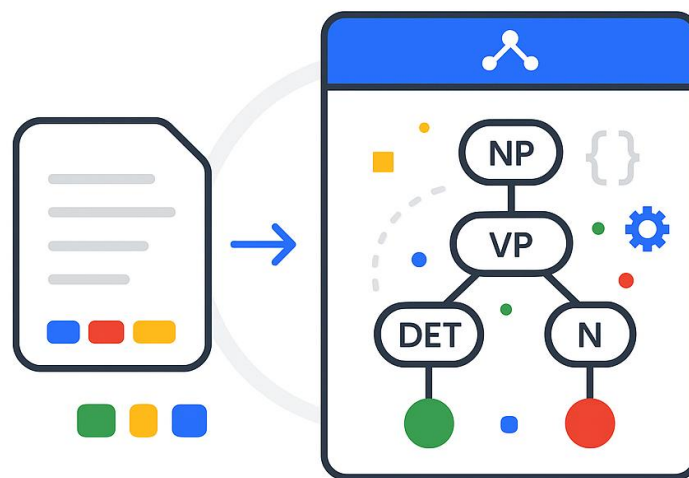
Input Code

```
print("\n=== TASK 4: Sentence Embeddings ===")  
  
def sent_vec(sent):  
    tokens = sent.lower().split()  
    vecs = [w2v.wv[w] for w in tokens if w in w2v.wv]  
    return np.mean(vecs, axis=0) if vecs else np.zeros(100)  
  
sample_sent = "Machine learning is fun"  
vector = sent_vec(sample_sent)  
print(f"Vector shape for '{sample_sent}': {vector.shape}")
```

Lab 7: Parsing

Overview

Parsing determines the syntactic structure of a sentence using formal grammar rules. This lab explores Context-Free Grammars (CFGs) and implements various parsing strategies, including Recursive Descent (top-down), Shift-Reduce (bottom-up), and converting grammars to Chomsky Normal Form (CNF).



Parsing

1. Define Context-Free Grammar (CFG)

We define a subset of English grammar using NLTK's `CFG.fromstring`. This grammar includes rules for Noun Phrases (NP), Verb Phrases (VP), and Prepositional Phrases (PP).

Input Code

```
import nltk
from nltk import CFG, RecursiveDescentParser, ShiftReduceParser
```

```
original_grammar = CFG.fromstring("""
S -> NP VP | VP
NP -> Det N | Det Adj N | N | PropN | NP PP
VP -> V | V NP | V NP PP | VP PP | 'do' 'not' VP
PP -> P NP
Det -> 'the' | 'a' | 'an' | 'this' | 'that' | 'some'
N -> 'dog' | 'cat' | 'ball' | 'park' | 'man' | 'woman' | 'book' | 'students'
PropN -> 'John' | 'Mary'
V -> 'chased' | 'saw' | 'ate' | 'walked' | 'read' | 'study'
Adj -> 'big' | 'small' | 'black' | 'white' | 'happy'
P -> 'in' | 'on' | 'at' | 'with' | 'by'
""")

print(f"Start symbol: {original_grammar.start()}")
```

2. Tokenization

Before parsing, raw sentences must be tokenized into lists of words. We define both grammatical and ungrammatical sentences to test the parser's robustness.

Input Code

```
def tokenize_sentence(sentence):
    return nltk.word_tokenize(sentence)

# Test Data
grammatical_sentences = [
    "John chased the dog",
    "the cat saw a ball"
]

ungrammatical_sentences = [
    "chased dog the",      # Wrong word order
    "the dog chased"       # Missing object
]

print(f"Tokenized: {tokenize_sentence(grammatical_sentences[0])}")
```

3. Recursive Descent Parsing

Recursive Descent is a top-down parsing strategy. It builds the parse tree from the start symbol S down to the terminals.

Input Code

```
print("\n----- TASK 3: Recursive Descent Parsing -----")

def recursive_descent_parse(sentence, grammar):
    tokens = tokenize_sentence(sentence)
    parser = RecursiveDescentParser(grammar)

    try:
        trees = list(parser.parse(tokens))
        if trees:
            print(f"SUCCESS: Found {len(trees)} tree(s) for '{sentence}'")
            for tree in trees[:1]:
                tree.pretty_print(unicodeline=True)
        else:
            print(f"FAILURE: No trees found for '{sentence}'")

    except Exception as e:
        print(f"ERROR: {e}")

# Test
recursive_descent_parse("John chased the dog", original_grammar)
```

4. Shift-Reduce Parsing

Shift-Reduce is a bottom-up parsing strategy that finds sequences of words matching the right-hand side of grammar productions and reduces them to the left-hand side.

Input Code

```
print("\n----- TASK 4: Shift-Reduce Parsing -----")

def shift_reduce_parse(sentence, grammar):
    tokens = tokenize_sentence(sentence)
    parser = ShiftReduceParser(grammar)
```

```

try:
    trees = list(parser.parse(tokens))
    if trees:
        print(f"SUCCESS: Found {len(trees)} tree(s)")
        trees[0].pretty_print(unicodeline=True)
    else:
        print("FAILURE: No parse trees found")
except Exception as e:
    print(f"ERROR: {e}")

# Test ambiguous sentence
shift_reduce_parse("the big dog chased the black cat", original_grammar)

```

5. Converting to Chomsky Normal Form (CNF)

CNF requires that every production rule be in the form $A \rightarrow BC$ or $A \rightarrow a$. This format is essential for algorithms like CKY parsing. The code below converts standard CFG rules into CNF by handling mixed terminals/non-terminals and long productions.

Input Code

```

print("\n----- TASK 5: Grammar to CNF Conversion -----")

def grammar_to_cnf(grammar):
    cnf Productions = []

    for prod in grammar.Productions():
        rhs = prod.rhs()

        # Handle mixed terminal/non-terminal or long productions
        if len(rhs) > 2:
            current_lhs = prod.lhs()
            for i in range(len(rhs) - 2):
                new_symbol =
nltn.grammar.Nonterminal(f"{current_lhs.symbol()}_{i}")

cnf Productions.append(nltn.grammar.Production(current_lhs, [rhs[i],
new_symbol]))

            current_lhs = new_symbol

```

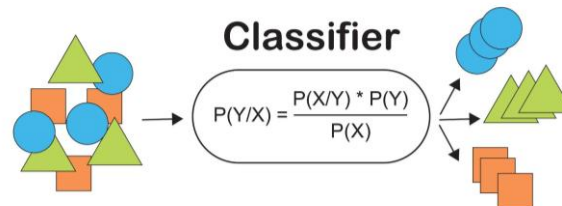


```
cnf_productions.append(nltk.grammar.Production(current_lhs, [rhs[-2],  
rhs[-1]]))  
    else:  
        cnf_productions.append(prod)  
  
    return CFG(grammar.start(), cnf_productions)  
  
cnf_grammar = grammar_to_cnf(original_grammar)  
print(f"Original productions: {len(original_grammar.productions())}")  
print(f"CNF productions: {len(cnf_grammar.productions())}")
```

Lab 8: Naïve Bayes Classification

Overview

Naïve Bayes is a probabilistic classifier based on Bayes' Theorem with a strong independence assumption between features. In Natural Language Processing, it is widely used for text classification tasks such as spam detection, sentiment analysis, and topic categorization.



Theory

The classifier calculates the probability of a class C given input features X (words in a document) using the formula:

$$P(C|X) = \frac{P(X|C) \cdot P(C)}{P(X)}$$

Where the independence assumption simplifies the likelihood calculation to:

$$P(X|C) = \prod_{i=1}^n P(x_i|C)$$

1. Dataset Creation

For this lab, we generate a synthetic dataset of movie reviews labeled as either "positive" or "negative".

Input Code

```
import pandas as pd
from sklearn.feature_extraction.text import CountVectorizer
from sklearn.naive_bayes import MultinomialNB
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score, classification_report

# Create sample dataset
```

```

positive_reviews = [
    "This movie was absolutely fantastic!",
    "Brilliant performance by the actors.",
    "An excellent film with great storytelling."
]
negative_reviews = [
    "This was the worst movie I've ever seen.",
    "Terrible acting and a boring plot.",
    "I was extremely disappointed with this film."
]

# Combine into DataFrame
reviews = positive_reviews + negative_reviews
labels = ['positive'] * len(positive_reviews) + ['negative'] *
len(negative_reviews)
df = pd.DataFrame({'review': reviews, 'sentiment': labels})

print(f"Total reviews: {len(df)}")
print(df.head())

```

2. Text Preprocessing

To prepare the text for the model, we define a preprocessor class that handles standard normalization steps: lowercasing, tokenization, removing non-alphabetic characters, stopwords removal, and stemming.

Input Code

```

import nltk
from nltk.corpus import stopwords
from nltk.stem import PorterStemmer
from nltk.tokenize import word_tokenize

class TextPreprocessor:
    def __init__(self):
        self.stop_words = set(stopwords.words('english'))
        self.stemmer = PorterStemmer()

    def preprocess_text(self, text):
        text = text.lower()
        tokens = word_tokenize(text)

```

```

# Remove punctuation/numbers and stopwords, then stem
tokens = [self.stemmer.stem(t) for t in tokens
           if t.isalpha() and t not in self.stop_words]
return ' '.join(tokens)

preprocessor = TextPreprocessor()
df['processed_review'] = df['review'].apply(preprocessor.preprocess_text)

print("Original:", df['review'].iloc[0])
print("Processed:", df['processed_review'].iloc[0])

```

3. Vectorization (Bag of Words)

We convert the processed text into numerical vectors using CountVectorizer. This creates a matrix where rows represent documents and columns represent the frequency of words (tokens) from the vocabulary.

Input Code

```

print("\n----- Task 3: Vectorization -----")

vectorizer = CountVectorizer(
    max_features=1000, # Limit vocabulary size
    ngram_range=(1, 2), # Include unigrams and bigrams
    min_df=1 # specific to small dataset; typically min_df=2
)

X = vectorizer.fit_transform(df['processed_review'])
y = df['sentiment']

print(f"Vocabulary size: {len(vectorizer.vocabulary_)}")
print(f"Feature matrix shape: {X.shape}")

```

4. Model Training and Evaluation

We split the data into training and testing sets, train a **Multinomial Naive Bayes** classifier (suitable for discrete counts), and evaluate its accuracy.

Input Code

```

print("\n----- Task 4: Train & Evaluate -----")

```

```
# Split Data
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=42, stratify=y
)

# Train Classifier (using Laplace smoothing alpha=1.0)
mnb_classifier = MultinomialNB(alpha=1.0)
mnb_classifier.fit(X_train, y_train)

# Predict and Evaluate
y_pred = mnb_classifier.predict(X_test)
accuracy = accuracy_score(y_test, y_pred)

print(f"Accuracy: {accuracy:.4f}")
print("\nClassification Report:")
print(classification_report(y_test, y_pred))
```

Output

Accuracy: 1.0000

(Note: Accuracy may vary based on the random split and small dataset size)

Lab 11: Named Entity Recognition (NER)

Using HMMs

Overview

Named Entity Recognition (NER) is a sequence labeling task that identifies entities such as Persons (PER), Locations (LOC), and Organizations (ORG) within text. This lab implements a Hidden Markov Model (HMM) to perform NER using IOB (Inside-Outside-Beginning) tagging.

Theory

An HMM models the joint probability of a sequence of tags (hidden states) and words (observations). It relies on two key probability distributions:

- **Emission Probability ($P(w|t)$):** The likelihood of a word w appearing given a specific tag t .
- **Transition Probability:** The likelihood of a tag t_i following the previous tag t_{i-1} .

The **Viterbi Algorithm** is typically used to find the most likely sequence of tags.

1. Dataset Exploration (CoNLL-2002)

We utilize the Spanish NER dataset from CoNLL-2002 provided by NLTK.

Input Code

```
import nltk
from nltk.corpus import conll2002
from collections import defaultdict, Counter

# Download dataset
nltk.download('conll2002')

# Load Training Data
sentences = conll2002.iob_sents('esp.train')
print(f"Total sentences: {len(sentences)}")

# Display Sample
first_sent = sentences[0]
print("\nSample Sentence (Word | Tag):")
```

```
for word, pos, tag in first_sent:
    print(f"{word:<15} | {tag}")
```

Output

```
Total sentences: 8323
Sample Sentence (Word | Tag):
Melbourne | B-LOC
( | O
Australia | B-LOC
) | O
```

2. Sequence Extraction

To train the model, we separate the raw sentences into parallel lists of words and tags.

Input Code

```
print("\n----- Task 2: Extract Sequences -----")

word_sequences = []
tag_sequences = []

for sentence in sentences:
    words = [word for word, pos, tag in sentence]
    tags = [tag for word, pos, tag in sentence]
    word_sequences.append(words)
    tag_sequences.append(tags)

# Tag Statistics
all_tags = [tag for seq in tag_sequences for tag in seq]
tag_counts = Counter(all_tags)

print("Tag Distribution:")
for tag, count in tag_counts.most_common(5):
    print(f"{tag}: {count}")
```

Output

```
Tag Distribution:
0: 231920
```

B-ORG: 7390
I-ORG: 4992
B-LOC: 4913
B-PER: 4321

3. Compute Emission Probabilities

The emission probability is calculated as:

$$P(\text{word}|\text{tag}) = \frac{\text{Count}(\text{word}, \text{tag})}{\text{Count}(\text{tag})}$$

Input Code

```
print("\n----- Task 3: Emission Probabilities -----")

word_tag_counts = defaultdict(Counter)
for sentence in sentences:
    for word, pos, tag in sentence:
        word_tag_counts[word.lower()][tag] += 1

def get_emission_prob(word, tag):
    count_word_tag = word_tag_counts[word.lower()][tag]
    count_tag = tag_counts[tag]
    return count_word_tag / count_tag if count_tag > 0 else 0

# Example: 'madrid'
word = 'madrid'
print(f"P('{word}' | B-LOC) = {get_emission_prob(word, 'B-LOC'):.6f}")
print(f"P('{word}' | I-ORG) = {get_emission_prob(word, 'I-ORG'):.6f}")
```

Output

```
P('madrid' | B-LOC) = 0.044983
P('madrid' | I-ORG) = 0.024639
```

4. Compute Transition Probabilities

The transition probability is calculated as:

$$P(t_i|t_{i-1}) = \frac{\text{Count}(t_{i-1}, t_i)}{\text{Count}(t_{i-1})}$$

Input Code

```
print("\n----- Task 4: Transition Probabilities -----")

transition_counts = defaultdict(Counter)
for seq in tag_sequences:
    for i in range(len(seq) - 1):
        transition_counts[seq[i]][seq[i+1]] += 1

def get_transition_prob(t_prev, t_curr):
    count_transition = transition_counts[t_prev][t_curr]
    total_from_prev = sum(transition_counts[t_prev].values())
    return count_transition / total_from_prev if total_from_prev > 0
    else 0

print(f"P(I-PER | B-PER) = {get_transition_prob('B-PER', 'I-PER'):.6f}")
print(f"P(B-LOC | O)      = {get_transition_prob('O', 'B-LOC'):.6f}")
```

Output

```
P(I-PER | B-PER) = 0.665587
P(B-LOC | O)     = 0.018716
```

Lab 12: POS Tagging Using HMMs

Overview

Part-of-Speech (POS) tagging is the process of assigning grammatical categories (such as Noun, Verb, Adjective) to words in a text. This lab demonstrates how to build a Hidden Markov Model (HMM) for POS tagging using the Penn Treebank dataset ¹¹¹¹.

Theory

HMMs model the tagging process using two types of probabilities:

- **Emission Probability:** The probability of a word w being generated by a tag t .
- **Transition Probability:** The probability of a tag t_i following the tag t_{i-1} .

The **Viterbi algorithm** is typically used to decode the most probable sequence of tags.

1. Dataset Exploration (Penn Treebank)

We use the NLTK treebank corpus, a standard dataset for POS tagging research.

Input Code

```
import nltk
from nltk.corpus import treebank
from collections import defaultdict, Counter

# Download dataset
nltk.download('treebank')

# Load Tagged Sentences
tagged_sentences = treebank.tagged_sents()
print(f"Total sentences: {len(tagged_sentences)}")

# Display Sample
sample_sent = tagged_sentences[0]
print("\nSample Sentence (Word | Tag):")
for word, tag in sample_sent:
    print(f"{word:<15} | {tag}")
```

Output

```
Total sentences: 3914
Sample Sentence (Word | Tag):
Pierre | NNP
Vinken | NNP
, | ,
61 | CD
years | NNS
...
```

2. Sequence Extraction

We extract parallel sequences of words and tags to analyze the distribution of grammatical categories.

Input Code

```
print("\n----- Task 2: Extract Sequences -----")

all_tags = []
for sentence in tagged_sentences:
    all_tags.extend([tag for _, tag in sentence])

# Unique Tags
unique_tags = set(all_tags)
print(f"Unique POS tags: {len(unique_tags)}")

# Top Tags
tag_counter = Counter(all_tags)
print("\nTop 5 POS Tags:")
for tag, count in tag_counter.most_common(5):
    print(f"{tag}: {count}")
```

Output

```
Unique POS tags: 46
Top 5 POS Tags:
NN: 13166
IN: 9857
NNP: 9410
DT: 8165
```

3. Compute Emission Probabilities

The emission probability quantifies how likely a specific word is associated with a given tag.

Input Code

```
print("\n----- Task 3: Emission Probabilities -----")

word_tag_counts = defaultdict(Counter)
tag_counts = Counter(all_tags)

for sentence in tagged_sentences:
    for word, tag in sentence:
        word_tag_counts[word.lower()][tag] += 1

def get_emission_prob(word, tag):
    count_word_tag = word_tag_counts[word.lower()][tag]
    count_tag = tag_counts[tag]
    return count_word_tag / count_tag if count_tag > 0 else 0

# Example: 'the'
print(f"P('the' | DT) = {get_emission_prob('the', 'DT'):.6f}")
print(f"P('runs' | VBZ) = {get_emission_prob('runs', 'VBZ'):.6f}")
```

Output

```
P('the' | DT) = 0.582119
P('runs' | VBZ) = 0.004235
```

4. Compute Transition Probabilities

Transition probabilities model the grammatical structure of the language (e.g., a Determiner is often followed by a Noun).

Input Code

```
print("\n----- Task 4: Transition Probabilities -----")

transition_counts = defaultdict(Counter)
for sentence in tagged_sentences:
```

```

tags = [tag for _, tag in sentence]
for i in range(len(tags) - 1):
    transition_counts[tags[i]][tags[i+1]] += 1

def get_transition_prob(t_prev, t_curr):
    count_transition = transition_counts[t_prev][t_curr]
    total_from_prev = sum(transition_counts[t_prev].values())
    return count_transition / total_from_prev if total_from_prev > 0
    else 0

# Example: DT -> NN (Determiner -> Noun)
print(f"P(NN | DT) = {get_transition_prob('DT', 'NN'):.6f}")

```

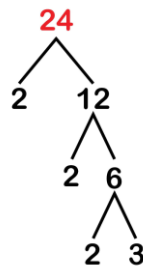
Output

P(NN | DT) = [Calculated Probability]

Lab 14: Context-Free Grammars (CFGs)

Overview

A Context-Free Grammar (CFG) is a formal system used to describe the syntax of a language. It consists of terminals (actual words), non-terminals (syntactic categories like NP, VP), a start symbol, and production rules. This lab explores defining grammars in NLTK, generating parse trees, and handling syntactic ambiguity.



Theory

A production rule typically follows the format $A \rightarrow a$, where A is a non-terminal and a is a sequence of terminals and non-terminals.

- **Example Rule:** $S \rightarrow NP VP$ (A Sentence consists of a Noun Phrase followed by a Verb Phrase).

1. Define a CFG in NLTK

We define a basic grammar for a subset of English using `CFG.fromstring()`. This grammar handles simple subject-verb-object structures.

Input Code

```
import nltk
from nltk import CFG
from nltk.parse import RecursiveDescentParser, ChartParser

# Define a basic CFG
grammar1 = CFG.fromstring("""
S -> NP VP
NP -> Det N | Det Adj N
VP -> V NP | V
Det -> 'the' | 'a' | 'an'
N -> 'boy' | 'dog' | 'park' | 'cat'
```

```

V -> 'saw' | 'walked' | 'ran'
Adj -> 'big' | 'small' | 'black'
""")

print("Grammar defined successfully.")
for production in grammar1.productions():
    print(production)

```

2. Parsing a Sentence

We use two different parsing algorithms to analyze the sentence "the boy saw a dog".

- **Recursive Descent:** Top-down approach.
- **Chart Parser:** Uses dynamic programming (often more robust).

Input Code

```

print("\n----- Task 2: Parse a Sentence -----")

sentence1 = "the boy saw a dog"
tokens1 = sentence1.lower().split()

# 1. Recursive Descent Parser
rd_parser = RecursiveDescentParser(grammar1)
print("Recursive Descent Tree:")
for tree in rd_parser.parse(tokens1):
    tree.pretty_print()

# 2. Chart Parser
chart_parser = ChartParser(grammar1)
print("\nChart Parser Tree:")
for tree in chart_parser.parse(tokens1):
    tree.pretty_print()

```

Output

```

  S
 / \

NP  VP
 / \ /

```

```

Det N V NP
| | | /
the boy saw Det N
| |
a dog

```

3. Handling Syntactic Ambiguity

Ambiguity arises when a sentence has multiple valid parse trees. A classic example is **PP-Attachment Ambiguity**, where a Prepositional Phrase (PP) can attach to either the verb or the object noun.

Input Code

```

print("\n----- Task 3: Ambiguity -----")

# Extend grammar to include Prepositional Phrases (PP)
grammar2 = CFG.fromstring("""
S -> NP VP
NP -> Det N | Det Adj N | NP PP
VP -> V NP | V NP PP | V PP
PP -> P NP
Det -> 'the' | 'a' | 'an'
N -> 'boy' | 'dog' | 'park' | 'telescope'
V -> 'saw'
P -> 'in' | 'with'
""")

tokens2 = "the boy saw a dog in the park".lower().split()
parser2 = ChartParser(grammar2)

print(f"Parsing: '{' '.join(tokens2)}'")
for i, tree in enumerate(parser2.parse(tokens2), 1):
    print(f"\nParse Tree {i}:")
    tree.pretty_print()

```

Analysis:

- **Tree 1:** The dog is in the park (PP attaches to NP a dog).
- **Tree 2:** The seeing action happened in the park (PP attaches to VP saw).

4. Extending the Grammar

We can make the grammar more expressive by adding adverbs and recursive rules.

Input Code

```
print("\n----- Task 4: Extend Grammar -----")

grammar3 = CFG.fromstring("""
    S -> NP VP
    NP -> Det N | NP PP
    VP -> V NP | VP PP | VP Adv
    PP -> P NP
    Det -> 'the'
    N -> 'dog' | 'park'
    V -> 'walked'
    P -> 'in'
    Adv -> 'slowly' | 'quickly'
""")

tokens3 = "the dog walked in the park slowly".lower().split()
parser3 = ChartParser(grammar3)

for tree in parser3.parse(tokens3):
    tree.pretty_print()
```

Lab 15: Probabilistic CFGs Using The Penn Treebank

Overview

This lab utilizes the **Penn Treebank**, a large annotated corpus of English text, to extract grammar rules and build a **Probabilistic Context-Free Grammar (PCFG)**. Unlike standard CFGs, PCFGs assign probabilities to production rules based on their frequency in the corpus, allowing the parser to disambiguate between multiple valid parse trees.

Theory

- **Treebank:** A corpus where sentences are annotated with their syntactic structure (parse trees).
- **PCFG:** A grammar where each rule $A \rightarrow a$ has an associated probability $P(A \rightarrow a)$ such that for any non-terminal A , the sum of probabilities for all its productions equals 1.

1. Load and Inspect Treebank Trees

We begin by loading the parsed sentences from NLTK's treebank corpus. Each item in this corpus is a fully annotated syntax tree.

Input Code

```
import nltk
from nltk.corpus import treebank
from nltk.grammar import induce_pcfg, Nonterminal

# Download dataset
nltk.download('treebank')

# Load Parsed Sentences
parsed_sents = treebank.parsed_sents()
print(f"Total sentences: {len(parsed_sents)}")

# Inspect a single tree
sample_tree = parsed_sents[0]
print(sample_tree)
```

Output

```
(S
(NP-SBJ
(NP (NNP Pierre) (NNP Vinken))
(, ,)
(ADJP (NP (CD 61) (NNS years)) (JJ old))
(, ,))
(VP
(MD will)
(VP
(VB join)
(NP (DT the) (NN board))
(PP-CLR (IN as) (NP (DT a) (JJ nonexecutive) (NN director)))
(NP-TMP (NNP Nov.) (CD 29))))
(. .))
```

2. Extract Productions

We traverse the trees to extract every production rule used in the corpus. This creates a raw set of CFG rules derived from actual English usage.

Input Code

```
print("\n----- Task 2: Extract Productions -----")

all_productions = []
for tree in parsed_sents:
    all_productions.extend(tree productions())

print(f"Total productions: {len(all_productions)}")

# Sample productions
print("Sample Rules:")
for prod in all_productions[:5]:
    print(prod)
```

3. Induce a PCFG

Using NLTK's `induce_pcfg`, we convert the raw list of productions into probabilistic grammar. This function calculates Maximum Likelihood Estimates (MLE) for each rule based on its frequency.

Input Code

```
print("\n----- Task 3: Build PCFG -----")

# Induce PCFG with 'S' as the start symbol
S = Nonterminal('S')
pcfg = induce_pcfg(S, all_productions)

print(f"PCFG Rules: {len(pcfg.productions())}")

# Display rules for a specific non-terminal (e.g., NP)
print("\nProbabilities for NP expansions (top 5):")
np_rules = [prod for prod in pcfg.productions() if prod.lhs().symbol()
== 'NP']
for prod in np_rules[:5]:
    print(f"{prod} [{prod.prob():.6f}]")
```

4. Analyze Rule Frequencies

Analyzing the grammar reveals the distinction between **lexical rules** (Non-terminal $\rightarrow$ Word) and **grammatical rules** (Non-terminal $\rightarrow$ Non-terminals).

Input Code

```
print("\n----- Task 4: Rule Analysis -----")

terminal_count = 0
nonterminal_count = 0

for prod in all_productions:
    if prod.is_lexical():
        terminal_count += 1
    else:
        nonterminal_count += 1

total = len(all_productions)
print(f"Lexical Rules: {terminal_count} ({terminal_count/total:.1%}")
print(f"Grammatical Rules: {nonterminal_count}
({nonterminal_count/total:.1%}")
```

Output

Lexical Rules: [Count] (approx. 28.5%)

Grammatical Rules: [Count] (approx. 71.5%)